holibob

General

Version: 4

11 February 2026

Table of Contents

1 Working together 1
2 Let's connect your systems with our GraphQL platform 1
3 This documentation 1
4 The Holibob API - Introduction 3
4.1 Introduction 3
4.2 Why GraphQL 4
4.3 Authentication 4
4.3.1 Generating the signature for each request 5
4.3.1.1 Example: 5
4.4 Naming conventions 6
4.5 The simplest query 7
4.5.1 Query 7
4.5.2 Responses 7
4.5.3 Matched Response 8
4.6 Rate Limits 9
4.7 Sandbox mode 10
5 The Holibob API - Look to Book Flow 11
5.1 Introduction 11
5.2 Fetch Product List - The basics 12
5.2.1 Query 12
5.2.2 Example Responses 13
5.3 Fetch Availability List for a Given Product 15
5.3.1 Methods 15
5.3.1.1 Deprecated Method 15
5.3.1.2 Recursive Method 16
5.3.1.3 Source-Specific Method 20
5.3.2 Validation 23
5.3.2.1 Query 23
5.3.2.2 Response 24
5.4 Fetch and respond to Options for a single Availability 24
5.4.1 Fetch the initial optionList 25
5.4.1.1 Initial Query 25
5.4.1.2 Initial Response 26
5.4.2 Setting the answer for the initial option(s) 27
5.4.2.1 Answering the first options group 27
5.4.2.2 Response informs options are NOT complete 27

5.4.3 Repeat the initial call for any further options ... 28
5.4.3.1 Query ... 28
5.4.3.2 Example Response ... 29
5.4.4 Setting the answers for any additional options ... 30
5.4.4.1 Query ... 30
5.4.4.2 Example Response ... 31
5.5 Fetch and respond to Pricing Categories for a single Availability ... 31
5.5.1 Fetch the Pricing Categories for the Avaialbility ... 32
5.5.1.1 Query ... 32
5.5.1.2 Example Response ... 32
5.5.2 Setting the units for each requested Pricing Category ... 34
5.5.2.1 Send the units to the availability ... 34
5.5.2.2 Example Response ... 35
5.6 Creating a booking ... 36
5.6.1 Create a new Booking ... 36
5.6.1.1 Mutation ... 36
5.6.1.2 Example Response ... 36
5.6.2 Create a new Booking with Consumer Trip ... 37
5.6.2.1 Mutation ... 37
5.6.2.2 Example Response ... 38
5.6.3 Add your Availability to your Booking ... 38
5.6.3.1 Request ... 38
5.6.3.2 Example Response ... 38
5.7 Input answers to the booking ... 39
5.7.1 Structure of a question list ... 40
5.7.2 Request ... 40
5.7.3 Example Response ... 40
5.7.4 Fetch the booking questions ... 41
5.7.4.1 Request ... 41
5.7.4.2 Example Response ... 42
5.7.5 Send the Answers for each Booking Question ... 45
5.7.5.1 Request ... 45
5.7.5.2 Example Response ... 46
5.8 Commit Booking ... 46
5.8.1 Commit the booking ... 47
5.8.1.1 Request ... 47
5.8.1.2 Example Response ... 47
5.8.2 Poll for the state of the Booking ... 48
5.8.2.1 Request ... 48
5.8.2.2 Example Response ... 48
5.8.3 Retrieve all bookings for ConsumerTrip or Consumer ... 49
5.8.3.1 Request ... 49

5.8.3.2 Example Response ... 50
6 The Holibob API - Product & ProductList ... 52
6.1 Fetch Product List - Product Content List ... 53
6.2 Fetch Product List - by Search ... 55
6.3 Fetch Product List - by Place ... 55
6.4 Fetch Product List - by isFeatured ... 56
6.5 Fetch Product List - by Geo Circle ... 56
6.6 Fetch Product List - by Geo Rectangle ... 57
6.7 Fetch Search Bar ... 58
6.8 Fetch Suggested Products for Product/ProductList ... 59
6.9 Rate Limits ... 60
6.10 Sandbox mode ... 61
7 The Holibob API - Product Discovery ... 63
7.1 Introduction ... 63
7.2 Design Principles ... 63
7.3 Inputs ... 64
7.4 Output ... 67
7.5 Examples ... 69
7.5.1 1. Recommended Destinations ... 69
7.5.2 2. Recommended Search Terms ... 70
7.5.3 3. Recommended Products ... 71
8 The Holibob API - Uploading file to booking question of type `file` ... 73
9 The Holibob API - Cancellation Policies ... 75
9.1 Introduction ... 75
9.2 Holibob API Relevant Fields ... 75
9.2.1 Effective Refund Amounts ... 75
9.2.2 Effective Cancellation Policy ... 75
9.2.3 Cancellation Penalty Lists ... 76
9.3 Understanding a Cancellation Policy ... 76
9.3.1 Cancellation Date Ranges ... 76
9.3.2 Refund Amount ... 77
9.3.3 Formatted Text ... 77
9.4 Examples ... 77
9.4.1 1. Percentage Refund Relative to the Travel Date ... 77
9.4.1.1 Data ... 77
9.4.1.2 Calculations ... 78

9.4.2 2. Absolute Refund Relative to the Booking Date ... 79
9.4.2.1 Data ... 79
9.4.2.2 Calculations ... 81
10 The Holibob API - Payment Flow ... 83
10.1 Introduction ... 83
10.2 Stripe Payment Flow ... 83
10.3 Integration Guide ... 84
10.3.1 Overview ... 84
10.3.2 Step 1: Fetching the Payment Intent ... 84
10.3.2.1 GraphQL Query ... 84
10.3.2.2 Parameters ... 84
10.3.2.3 Response ... 84
10.3.3 Step 2: Implementing the Payment Form ... 85
10.3.3.1 Prerequisites ... 85
10.3.3.2 Implementation Steps ... 85
10.3.4 Error Handling ... 87
10.3.4.1 Example Error Response ... 87
10.3.5 Sandbox vs Production ... 88
10.3.5.1 Testing ... 88
11 The Holibob API - Booking Questions ... 89
12 The Holibob API - Booking Cancellation ... 92
12.1 Recommended Implementation Steps ... 92
12.2 Retrieving Booking Cancellation Data ... 92
12.2.1 GraphQL Query Example ... 92
12.2.1.1 Query ... 92
12.2.1.2 Response ... 93
12.2.2 Key Fields Explained ... 94
12.3 bookingCancel Mutation ... 94
12.3.1 Input Parameters ... 94
12.3.1.1 Mutation ... 95
12.3.1.2 Response ... 95
12.3.2 Asynchronous Processing ... 95

We're committed to helping our partners integrate seamlessly with Holibob's systems to deliver world-class booking experiences for travellers.

The Holibob GraphQL API enables authorised partners to query for products, availability, and booking functionality through a single, flexible interface. Built using GraphQL, this API allows you to request exactly the data you need nothing more, nothing less streamlining integration and improving performance.

1 Working together

- Our GraphQL API supports querying and booking across experiences, hotels, and other product types.
- You bring the user interface and booking context we provide the structured data and business logic.

2 Let's connect your systems with our GraphQL platform

- **You do it** – Integrate directly using our documented authentication, queries, and response structures.
- **We support you** – Comprehensive guides, example payloads, and testing tools to get you up and running quickly.
- **Together** – Create intuitive, real-time booking journeys that enhance your customer experience.

3 This documentation

This documentation provides a complete overview of how to work with the Holibob GraphQL API. It includes key information to support your integration, covering core concepts, example queries, naming conventions, and practical guidance throughout.

You'll also find general information on how to access and authenticate with the API, as well as details on the environments available for development and production use. The sandbox environment is designed to support early testing, while the production environment is used for live traffic once your integration is ready.

Each section is intended to help you build, test, and launch a reliable connection, with clear examples and guidance along the way.

4 The Holibob API - Introduction

4.1 Introduction

This documentation covers the Holibob GraphQL API that allows authorised partners to query for products and associated availability and to create and confirm bookings for these availabilities.

GraphQL uses Queries, Mutations and Subscriptions to facilitate your interactions with the Holibob eCommerce engine for experiences.

> A query can look very similar to JSON but it is of note that elements are generally separated by a newline and there is no comma at the end of each line.

Before using any part of the API you will need to apply to Holibob for an appropriate `holibobApiKey` and `holibobSecret` that may be used to sign requests to the API. You will also be provided with a partner administration `username` and `password` that provide access to an administration portal "Extranet" and to any consumer-facing “white-label” websites that we operated on your behalf.

If you require additional users to be created, perhaps with limited permissions then our commercial team will be happy to facilitate this for you.

In the first instance, your channels will be restricted to sandbox mode to enable worry-free testing. You can read more about this in the section titled “Sandbox Mode”.

The focus of the development team has been to make the API as simple to integrate with as possible whilst still providing a rich interface for product content, filtering, booking and management of your data.

We are keen to work closely with you and will always ensure prompt responses to enquiries often leading to the setup of conference calls and screen-sharing sessions with our most experienced product managers and technical staff.

The Holibob GraphQL API allows authorised partners to query for products and their associated availability and to make bookings on behalf of their consumers.

The production API is available at: https://api.production.holibob.tech/graphql

4.2 Why GraphQL

We're using GraphQL because it's great for developers, partners, and merchants.

GraphQL brings:

- A richly expressive open-source specification, maintained by a dedicated foundation.
- Schemas and types that strictly define valid requests and responses.
- Introspection, meaning you can ask any GraphQL API what it can do.
- Developer tools to ease learning and integration.
- The flexibility to specify exactly what data you care about, and retrieve only as much or as little data as needed.
- The efficiency of combining multiple traditional REST endpoints into one request.
- The freedom to use any programming language you want — and no more SDKs to manage and update on your server.

With GraphQL, we are enabling faster, safer, and more extensible integration of commerce capabilities into your product

4.3 Authentication

To access the API, you will be provided with two separate but connected elements.

These are:

- **API-Key**: This must be passed in the header of all requests
- **API-Secret**: This must NOT be passed in the header. It must be used to generate an HMAC signature that will be passed in the header of all requests

The required request headers are:

- **X-API-Key**: The API-key provided to the partner by Holibob
- **X-Holibob-Date**: The date that the request was created in a simplified extended ISO format (ISO 8601).

- **X-Holibob-Signature**: An HMAC signature created using the API-Secret key provided to you.

4.3.1 Generating the signature for each request

You must create a function that will generate the `x-holibob-signature` and append it to the headers for every request that you send to the API.

To do this you must create a string that is the concatenation of each of the following items.

The signature is computed as the base64 encode of the SHA1 hash of the concatenated string.

The X-Holibob-Signature is created using the following technique:

1. **Create a string by concatenating the following:**
   a. **The Current Date** - as specified in the X-Holibob-Date header.
   b. **The API-Key** - as provided to you by Holibob
   c. **The Method** - in upper-case will be "POST".
   d. **The Path** - will always be "/graphql"
   e. **The Body** - of the request
2. **Hash the string with the SHA1 hash algorithm**
3. **Base64 encode the signature**
4. Append the resulting value to the headers of each request with the key: **X-Holibob-Signature**

> .You can verify that you have a valid signature by calling the `welcome` query. If your signature is valid the response will have an authStatus = "AUTHENTICATED" and other data in the response will confirm you PartnerName and other related details

The API call will fail if the signature is not correct, or if the date is too far in the past or future.

4.3.1.1 Example:

Let's say we're calling the activity search service. We want the results in English and prices in Icelandic Krona (ISK). We have the following:

- **X-Holibob-Date**: 2013-11-09T14:33.462Z
- **X-API-Key**: f9235a6a15c340b6b1e1cb5f3687d789 (this is not a real key and can not be used - you must use the key provided to you by Holibob)
- **Secret Key**: 23e2c7da7f7048e5b46f96bc91324800

Now, let's calculate the signature:

First, we concatenate the values into a canonical string in which the order of elements must be as described here:

- **Date** = "2013-11-09T14:33.462Z"
- **API-key** = "f9235a6a15c340b6b1e1cb5f3687d789"
- **Method** = "POST"
- **Path** = "/graphql"
- **Body** = "{variables: {}, query":"query holibob {\n welcome\n}"}"

We then use this canonical string, along with the Secret to calculate the SHA1 hash for the signature, and Base64 encode it:

- **x-holibob-signature**: HE39ukkpJXLzV+TjSi/HVYkAZ/Y=

4.4 Naming conventions

At Holibob we work tirelessly to maintain high standards across our systems and key to this are the conventions for naming entities and standardisation of syntax across the API

There are hundreds of different entities available via the API but many aspects of the API require enhanced permissions in order to access them. We are always happy to work with partners to extend access into all areas of the system as may be relevant to your particular use case.

As this document covers the most basic features relating to finding and booking products we will use only a very small number of endpoints and can refrain from introducing the more complete parts of the system.

We will be dealing with the following entities

- **Product** - a unique record for a single Experience, Hotel, Tour or Activity complete with all of its related metadata
- **Availability** - a unique record representing the availability to you of a Product on a given day complete with details of any options that may exist
- **OpinionatedAvailability** - The same unique Availability detailed above but after you have provided selections for all options and pricingCategories

- **Booking** - A unique "shopping cart" to which one or more OpinionatedAvailability may be added. A Booking will have a unique ID and you may also append your own key `partnerExternalReference`

Queries are available to request either a single record or a list of records. The structures available for a given item will always be the same regardless of whether you request a single item or a list of items.

For example:

- You may call for `productList` passing one or more filters such as a City, Category, Curation or many others. The response will contain an array of `product` that match the filter criteria. Each item in the list will have a unique UUID
- You may call for `product` passing the known ID for the given product. The response will contain a single Product

4.5 The simplest query

The simplest query that the API supports is the `welcome` query which can be run at any time in order to confirm that you have successfully connected to the API.

4.5.1 Query

welcome query

```
query {
  welcome
}
```

4.5.2 Responses

Unauthenticated Response

If you have not provided credentials by signing the request then the response will be similar to:

```
{
    "data": {
        "welcome": {
            "message": "Welcome to Holibob API",
            "systemTimestamp": "Thu Apr 06 2023 10:08:12 GMT+0000 (Coordinated Universal
Time)",
            "authStatus": "UNAUTHENTICATED",
        }
    }
}
```

Authenticated Response

If you have provided credentials by signing the request then the response will be similar to:

```
{
  welcome:{
    message:"Welcome to Holibob API (SANDBOX)"
    systemTimestamp:"Thu Apr 06 2023 10:04:10 GMT+0000 (Coordinated Universal Time)"
    authStatus:"AUTHENTICATED"
    partnerId:"<basedOnYourCredenials>"
    partnerName:"<basedOnYourCredenials>"
    partnerChannelId:"<basedOnYourCredenials>"
    partnerChannelName:"<basedOnYourCredenials>"
    allowedCurrencies:[
      0:"GBP"
      1:"EUR"
      2:"USD"
      4:"SAR"
    ]
  }
}
```

Matched Response

4.5.3 Matched Response

It is also possible to send a request to some public queries (Including welcome) and pass the `origin` on the header.

In this case, the response will partially authenticate to a `MATCHED` state and will be similar to:

```
{
        "welcome": {
            "message": "Welcome to Holibob API",
            "systemTimestamp": "Thu Apr 06 2023 10:12:01 GMT+0000 (Coordinated Universal
Time)",
            "authStatus": "MATCHED",
            "dbStage": "production",
            "saleCurrency": null,
            partnerId:"<basedOnYourCredenials>"
            partnerName:"<basedOnYourCredenials>"
            partnerChannelId:"<basedOnYourCredenials>"
            partnerChannelName:"<basedOnYourCredenials>"
            "allowedCurrencies": [
                "EUR",
                "GBP",
                "USD"
            ]
        }
    }
}
```

4.6 Rate Limits

Your holibob API key is associated with a usage plan that provides rate limits and quotas to protect you and us from bad actors or run-away code.

If you receive a “Limit Exceeded” error and believe this to be an error then you may contact holibob and request a rate limit increase.

The default limits are:

- 2 requests per second with a burst capability of 10 requests per second
- 2,500 requests per day

4.7 Sandbox mode

The credentials initially supplied will be valid for the production environment but limited to the "Sandbox" mode. We strongly encourage you to do all initial development against the production URL as detailed above.

In the API this can be confirmed by examination of the response from the welcome query as the isSandbox flag will be set to true.

On white-label and agent portals, you will also see an info icon on your account button.

Whilst in this mode, you are free to use the production API and portals for all testing activities including the creation and confirmation of bookings. These bookings will automatically be sandboxed and as such, they will not be confirmed with suppliers and you will not incur any charge for the bookings. All functionality is available in this state and you will be gaining access to production product and live availabilities.

All testing is therefore done against the production system ensuring that all code you write is immediately production ready.

Once you have completed initial integrations you will be able to apply to the Holibob commercial team to have the credentials upgraded to allow live bookings. **Once updated, all bookings confirmed on the production API will be live, you will be charged and the booking will be confirmed with the supplier.**

You will be able to see all bookings within the Extranet when logged in with credentials linked to your partner account. When using Extranet you can switch between viewing sandboxed or live bookings.

There is an additional API endpoint available that is simply an alias against the production API, however, this API will enforce sandbox mode regardless of the state of your account

The sandbox API is available at: https://api.sandbox.holibob.tech/graphql

5 The Holibob API - Look to Book Flow

5.1 Introduction

This documentation covers the parts of the Holibob API required to discover, configure and book a single product availability.

The sections that follow detail the simplest API calls necessary to:-

- **Discover Products -** You will use the `producList` query along with filters and pagination to present the consumer with products that match their searches.
- **Display Product Details** - You will use the `product` query to retrieve more data about a single product that the consumer has chosen to view.
- **Request Availability List** - You will use the `availabilityList` query passing a product Id and date range and to discover the dates on which the Product is available to book.
- **Discover and Set Availability Options** - You will use the `availability` query passing the ID that relates to the consumer's selected date in order to discover details of any options that the consumer must select and then provide the system with their chosen responses.
  - **StartTime**: You may be offered different time for the day at which the product is available.
  - **Guide Language**: for products that include a guide you may be presented with different languages that they can speak.
  - **A variant of the product**: You may be offered different variants of the product such as “Standard” or “Premium”.
  - **… more**: there is no hard limit on the number of options for any given product, you must provide a selection for each option.
- **Discover and Set Availability Pricing Categories** We will again use the `availability` query passing the ID to discover and set the number of pax for each price category such as Adult, Child etc.
- **Create a Booking** - You will use the `bookingCreate` mutation to create a "cart" so we can add one or more availabilities to this Booking.
- **Add our configured Availability to the Booking -** You will use the `bookingAddAvailability` mutation to add the above Availability to the created Booking.
- **OPTIONALLY** - You will allow the user to discover further ProductAvailabilities and add these to the same Booking prior to checkout.

- **Checkout** - You will use the `bookingConfirm` mutation to finalise the Booking. At this time the system will confirm the booking with the supplier and, depending on configuration, either we or the system must send confirmation emails and vouchers to the consumer.
- **Retrieve the booking** - You can use the `booking` query to request all of the information relating to the finalised booking including any details you may wish to send to the consumer including vouchers, tickets and all related Product details.
- **Retrieve all bookings for ConsumerTrip or Consumer -** You can use the `bookingList` query to filter bookings by a specific Consumer Trip or Consumer

It is possible to use the `booking` query passing the bookingId to retrieve all information for vouchers and tickets. Be aware however that there is a short period after using the `bookingConfirm is which the booking will be in a PENDING state whilst all checks are completed with the supplier. We will poll the booking query until this state changes

In this document, we use the format `$variableName` to indicate some value that you may be managing in the state either as a direct input from the consumer or from the session that the consumer is part of.

Typically these state values will have been determined by a previous interaction with the API.

5.2 Fetch Product List - The basics

The `productList` query is used to obtain a list of Products based on filters and pagination

Each product will have a unique `id` that you will use later to retrieve its Availabilities.

5.2.1 Query

productList query

```
query {
  productList(
    filter: {placeName: "Edinburgh"},
  )
```

```
  {
    nodes {
      id
      name
      guidePriceFormattedText
    }
  }
}
```

There are several hundred values that may be retrieved for a Product. These values are nested. As is always the case with GraphQL the structure of the data returned in the response is exactly the structure that you asked for in the request.

5.2.2 Example Responses

productList example response

```
{
  productList:{
    nodes:[
      {
        id:"e33751d7-485b-44a4-ad8f-2637ff749353"
        name:"Walk Through Test Product"
        guidePriceFormattedText:NULL
      },
      {
        id:"f3ce919a-fee1-11e9-b729-06829e32ec62"
        name:"JK Rowling's Edinburgh & the writing of Harry Potter"
        guidePriceFormattedText:NULL
      },
      {
        id:"595ba897-e850-4264-a677-804bc1c71935"
        name:"Cycle tour to the coast"
        guidePriceFormattedText:"€ 49.00"
      },

      ... etc

    ]
  }
}
```

In this simple example, we passed a filter for `PlaceName="Edinburgh"` and so the system returned a list of products that match that filter.

It is possible to pass many different filters and combine them.

The example asked only for `id`, `name` and `guidePriceFormattedText` and so that is what was returned.

There are however several hundred other attributes of the Product that may have been requested. Note too that `guidePrice` and `guidePriceCurrency` could have been asked for separately and that the `guidePriceFormattedText` is provided only for convenience.

Guide Prices

The `guidePrice` will be an APPROXIMATE Adult price for the next nearest availability of the product and is provided only as a guide and a from price. The true price of a product may vary by date and other criteria and will be returned only when a call is made to the `availabilityList` query.

Images

Each product will have one o more images available as may be requested with the `imageList` property inside of the product nodes. Each image has a unique ID. You can request the image from our global CDN in any format (WebP, PNG, JPG etc) and at any resolution. It is also possible to specify the aspect ration that you will present the image in. The CDN will perform any transformations required and return an image in exactly the resolution, format and shape required by your consumers.

Descriptions

Each product will have a description and may have many other elements of content such as inclusions, exclusions and itinerary. Each content item may be requested in one of nine supported languages including Arabic and may be requested in either PLAIN_TEXT, HTML or JSON format

5.3 Fetch Availability List for a Given Product

This query returns a list of available dates for a specific product.

5.3.1 Methods

We offer three methods for interacting with our `availabilityList` query:

Method	Benefits	Cons
Deprecated deprecated		▪ Not supported by all sources: • `SevenRooms`, `LightSpeed Golf`, `Expedia`
Recursive recommended	▪ Only write your client code once	▪ Complexity ▪ Requires multiple calls
Source-Specific	▪ Simple ▪ Requires a single call	▪ Must write separate client code for each source

5.3.1.1 Deprecated Method

deprecated

> ⚠ We continue to support this method but recommend adopting the **Recursive Method**. This updated approach offers greater robustness and versatility, tailored to the different requirements of our aggregated products.

Input

- `productId` : ID of the product
- `filter` :
  - `startDate` : filter by days on or after this date
  - `endDate` : filter by days on or before this date

Query

query

```
query {
  availabilityList(
```

```
    productId: "$productId",
    filter: {
      startDate: "$startDate",
      endDate: "$endDate",
    }
  )
  {
  nodes {
    id
    date
    guidePriceFormattedText
  }
}
```

Response

response

```
availabilityList:{
  nodes:[
    {
      id:"2baea4d1-1bd8-44e9-b948-ba4a5cf8dd9f",
      date:"2023-04-06",
      soldOut:false,
      guidePriceFormattedText:"€ 206.00",
    },
    {
      id:"9e2bb7f0-cb8e-4cad-935a-20dd27d194c6",
      date:"2023-04-07",
      soldOut:false,
      guidePriceFormattedText:"€ 206.00",
    }
  ]
}
```

5.3.1.2 Recursive Method

recommended

The recursive nature of this query pattern makes it future-proof. It can accommodate any additional options that may be added in the future without requiring new code to be written. This is the recommended approach for interacting with the `availabilityList` query.

Initially, the query returns a set of options that must be answered before the available dates can be returned. With each iteration, new options may be generated based on previous answers. Once all options are answered, the query returns the available dates.

Input

- `productId` : ID of the product
- `sessionId` : UUID used to store the current progress of chosen options per *session.* Generated on the first iteration, to be used on subsequent calls
- `optionList` :
  - `id` : ID of the option
  - `value` : answer value of option

Query 1

query

```
query Demo {
    availabilityList(
        productId: "31293efb-e8f3-11e9-b729-06829e32ec62"
    ) {
        sessionId
        nodes {
            id
            date
        }
        optionList {
            nodes {
                id
                label
                value
                required
                type
```

```
                dataType
            }
        }
    }
}
```

Response 1

response

```
{
  sessionId: "1235678-1234-1234-1234-123456789123",
  nodes: null,
  optionList: [
  {
      id: "id-1",
      label: "Start Date",
      value: null,
      required: true,
      type: "START_DATE",
      dataType: "DATE"
    },
    {
      id: "id-2",
      label: "End Date",
      value: null,
      required: true,
      type: "END_DATE",
      dataType: "DATE"
    }
  ]
}
```

Query 2

query

```
query Demo {
    availabilityList(
        productId: "31293efb-e8f3-11e9-b729-06829e32ec62"
        sessionId: "1235678-1234-1234-1234-123456789123"
```

```
        optionList: [
            { id: "id-1", value: "2020-04-01" },
            { id: "id-2", value: "2020-04-03" },
        ]
    ) {
        sessionId
        nodes {
            id
            date
        }
        optionList {
            nodes {
                id
                sessionId
                label
                value
                required
                type
                dataType
            }
        }
    }
}
```

Response 2

response

```
{
  sessionId: "1235678-1234-1234-1234-123456789123",
  nodes: [
      {
        id: "availability-list-item-1",
        date: "2020-04-01"
      },
      {
        id: "availability-list-item-2",
        date: "2020-04-02"
      },
      {
        id: "availability-list-item-3",
        date: "2020-04-03"
```

```
        }
    ],
    optionList: [
    {
        id: "id-1",
        label: "Start Date",
        value: "2020-04-01",
        required: true,
        type: "START_DATE",
        dataType: "DATE"
      },
      {
        id: "id-2",
        label: "End Date",
        value: "2020-04-03",
        required: true,
        type: "END_DATE",
        dataType: "DATE"
      }
    ]
  }
```

5.3.1.3 Source-Specific Method

> The source-specific query pattern offers the ability to address all potential questions within a single call of the `availabilityList` query. Unlike the recursive query, this pattern aims for efficiency and simplicity by answering all options upfront.

However, it comes with its own set of considerations:

1. **Predefined Options:** To utilise the source-specific query pattern, you must know all available options and relevant fields beforehand. This changes based on the source of the product.
2. **Brittleness:** The source-specific pattern's dependence on predefined options makes it more brittle. Any changes or additions to these options will require code modifications.

Input

- `productId` : ID of product to book
- `optionList` :
  - `id` : client-generated ID for the question

- `value` : answer of option
- `type` : identifier for the type of question

Below is a sample of our source-specific flow using default options, designed to cover the majority of our product types. Should you need integration support with the `LightSpeed Golf`, `Expedia`, or `SevenRooms` sources, please connect with your dedicated account manager for specific documentation.

Query

query

```
query Demo {
    availabilityList(
        productId: "31293efb-e8f3-11e9-b729-06829e32ec62"
        optionList: [
            { id: "$clientGeneratedID-2", value: "2020-04-01", type: "START_DATE" },
            { id: "$clientGeneratedID-3", value: "2020-04-03", type: "END_DATE" },
        ]
    ) {
        sessionId
        nodes {
            id
            date
        }
        optionList {
            nodes {
                id
                label
                value
                required
                type
                dataType
            }
        }
    }
}
```

Response

response

```
{
  sessionId: "1235678-1234-1234-1234-123456789123",
  nodes: [
      {
        id: "availability-list-item-1",
        date: "2020-04-01"
      },
      {
        id: "availability-list-item-2",
        date: "2020-04-02"
      },
      {
        id: "availability-list-item-3",
        date: "2020-04-03"
      }
  ],
  optionList: [
  {
      id: "$clientGeneratedID-2",
      label: "Start Date",
      value: "2020-04-01",
      required: true,
      type: "START_DATE",
      dataType: "DATE"
    },
    {
      id: "$clientGeneratedID-3",
      label: "End Date",
      value: "2020-04-03",
      required: true,
      type: "END_DATE",
      dataType: "DATE"
    }
  ]
}
```

5.3.2 Validation

Each option will undergo generic validation based on its **dataType**, along with additional specific validations dictated by source rules. Examples include limits such as the maximum number of people or the minimum number of rooms. Validation errors are then exposed through the **errorList** field in the query shown below.

5.3.2.1 Query

query

```
query Demo {
    availabilityList(
        productId: "31293efb-e8f3-11e9-b729-06829e32ec62"
        sessionId: "$clientGeneratedID-1" // used as cache key
        optionList: [
            { id: "$clientGeneratedID-2", value: "true", type: "START_DATE", parentId:
null},
            { id: "$clientGeneratedID-3", value: "55", type: "END_DATE", parentId: null},
        ]
    ) {
        id
        nodes {
            id
            date
        }
        optionList {
            nodes {
              errorList {
                nodes
              }
            }
        }
    }
}
```

5.3.2.2 Response

response

```
{
  id: "1235678-1234-1234-1234-123456789123",
  nodes: null,
  optionList: {
    nodes: [
      {
        errorList: {
          nodes: [
            "Error: The value 'true' provided for the START_DATE option is not a valid date
format."
          ]
        }
      },
      {
        errorList: {
          nodes: [
            "Error: The value '55' provided for the END_DATE option is not a valid date
format."
          ]
        }
      }
    ]
  }
}
```

5.4 Fetch and respond to Options for a single Availability

Each Availability returned in the previous query has a unique id and relates to a given date in the calendar. Dates that are not returned have zero availability and must not be offered to the consumer. These are cache values and are only guaranteed to live for 24-hours which is typically a lot longer than would ever be required.

You will use the `availability` query to both retrieve details of the availability AND update details of the availability.

> Using a query to mutate data in a GraphQL API is unusual but not outside of the specification. The pattern is used only in this part of the API as it is significantly easier to work with that using a combination of queries and mutations as is common elsewhere.

You must make **iterative calls** to the `availability` query as you inform the system of choices the consumer has made for

- **options** - returned to you as an array of questions each with an array of possible answers
- **pricingCategories -** returned to you as an array of categories containing details of pricing, restrictions such as `maxAllowed` and interdependencies such as the required ratio of Adult to Child

You are required to present the options to the consumer and submit their responses to the system. Each submission of an `option` or `pricingCategory` response may result in changes to other options already presented or yet to be discovered.

It is only possible to add the Availability to a Booking once all required options have been submitted and pricingCategory units set

5.4.1 Fetch the initial optionList

5.4.1.1 Initial Query

Fetch the option list from the availability query

```
query walkThrough {
  availability(id: "{{ state.availabilityId }}") {
    id
    optionList {
      nodes {
        id
        label
        dataType
        dataFormat
        availableOptions {
```

```
            label
            value
          }
          answerValue
          answerFormattedText
        }
      }
    }
  }
```

5.4.1.2 Initial Response

Response containing the initial option list

```
{
  "availability": {
    "id": "2baea4d1-1bd8-44e9-b948-ba4a5cf8dd9f",
    "optionList": {
      "nodes": [
        {
          "id": "12a06400-b7ed-4a70-ac0f-7e95258b465c",
          "label": "Ticket Type",
          "dataType": "OPTIONS",
          "dataFormat": null,
          "availableOptions": [
            {
              "label": "Premium",
              "value": "ffcdea9d-acd3-4c38-b49a-4f8b42bca0f8"
            },
            {
              "label": "Standard",
              "value": "f3ef9c0d-e8f4-4597-8ef6-8418a0a3ea73"
            }
          ],
          "answerValue": null,
          "answerFormattedText": null
        }
      ]
    }
  }
}
```

5.4.2 Setting the answer for the initial option(s)

In the above, there is a single option requiring the consumer to select either the Premium or Standard tour. You must present this option to the consumer and require a response. When the user has made their choice you will call the query again as below to change the data in the availability.

5.4.2.1 Answering the first options group

Send the initial response(s) to the system

```
query {
  availability(id: $availabilityId,
    input: {
      optionList:
            [
        {
          id: "c0e212a6-3247-472e-9939-ecd79d180a2f",
          value: "4c4f056f-7a28-4955-811d-5cd1786619ee"
        }
      ]
    }
  ) {
    optionList {
      isComplete
    }
  }
}
```

5.4.2.2 Response informs options are NOT complete

Check the status of isComplete

```
{
    "data": {
        "availability": {
            "optionList": {
```

```
                "isComplete": false
            }
        }
    }
}
```

We will see that the response to `incomplete` is FALSE and so whilst we did answer all the options that were initially presented we an now assume that there is at least one new option that must be iterated.

In this case we chose a particular version of the Product and so now the system can present us with the startTime options

5.4.3 Repeat the initial call for any further options

5.4.3.1 Query

Fetch the optionList again

```
query {
  availability(id: $availabilityId) {
    id
    optionList {
      nodes {
        id
        label
        dataType
        dataFormat
        availableOptions {
          label
          value
        }
        answerValue
        answerFormattedText
      }
    }
  }
}
```

5.4.3.2 Example Response

Response with more options


```
{
    "data": {
        "availability": {
            "id": "711a0648-215e-43f5-9d17-8a376b62b92e",
            "optionList": {
                "nodes": [
                    {
                        "id": "6358fbc1-6db7-4363-ab1d-c4b2bf232fe1",
                        "label": "Ticket Type",
                        "dataType": "OPTIONS",
                        "dataFormat": null,
                        "availableOptions": [
                            {
                                "label": "Premium",
                                "value": "ffcdea9d-acd3-4c38-b49a-4f8b42bca0f8"
                            },
                            {
                                "label": "Standard",
                                "value": "f3ef9c0d-e8f4-4597-8ef6-8418a0a3ea73"
                            }
                        ],
                        "answerValue": "f3ef9c0d-e8f4-4597-8ef6-8418a0a3ea73",
                        "answerFormattedText": "Standard"
                    },
                    {
                        "id": "9c35c8b0-5aa6-46d4-af15-af51186881f6",
                        "label": "Start Time",
                        "dataType": "OPTIONS",
                        "dataFormat": null,
                        "availableOptions": [
                            {
                                "label": "09:00",
                                "value": "9f25d83f-1f64-4f7b-b99e-600f290d5e89"
                            },
                            {
                                "label": "11:00",
                                "value": "16bc80e9-40c3-4580-9647-1f5da3228413"
                            },
                            {
                                "label": "17:00",
```

```
                                "value": "8df28cb8-16bc-4dbf-a962-d3858786a373"
                            }
                        ],
                        "answerValue": null,
                        "answerFormattedText": null
                    }
                ]
            }
        }
    }
}
```

We will confirm at lines 24 and 25 that our initial option choice has been added to the Availability but observe at line 28 there is a new option for Start Time

We must again therefore submit the response to this option

5.4.4 Setting the answers for any additional options

5.4.4.1 Query

Sent the response for the Start Time options

```
query holibob($availabilityId: String!) {
  availability(id: $availabilityId,
    input: {
      optionList:
            [
        {
          id: "9c35c8b0-5aa6-46d4-af15-af51186881f6",
          value: "16bc80e9-40c3-4580-9647-1f5da3228413"
        }
      ]
    }
  ) {
    optionList {
      isComplete
    }
```

```
    }
}
```

5.4.4.2 Example Response

check the completion state of the availability

```
{
    "data": {
        "availability": {
            "optionList": {
                "isComplete": true
            }
        }
    }
}
```

Great! we now receive confirmation that `isComplete` = true and so we have satisfied all options.

Of course, this may not have been the case. It is dependent on each product exactly how many times this process may require to be iterated.

5.5 Fetch and respond to Pricing Categories for a single Availability

Now that we have confirmation that all options are satisfied it is possible to request the pricingCategory details for the Availability.

> Requesting pricingCategories for an Availabilirty that we have not satisfied all options on will always return a null pricingCategory node. This is because the available pricingCategories are dependant on the options

5.5.1 Fetch the Pricing Categories for the Avaialbility

5.5.1.1 Query

Fetch the initial pricingCategories for the Availability

```
query {
    availability(id: $availabilityId) {
        id
        maxParticipants
        minParticipants
        pricingCategoryList {
            nodes {
                id
                label
                minParticipants
                maxParticipants
                maxParticipantsDepends {
                    pricingCategoryId
                    multiplier
                    explanation
                }
                units
                unitPrice {
                    netFormattedText
                    grossFormattedText
                }
                totalPrice {
                    grossFormattedText
                }
            }
        }
    }
}
```

5.5.1.2 Example Response

Check the pricingCategories available and the prices and restrictions that apply

```
{
    "data": {
        "availability": {
            "id": "711a0648-215e-43f5-9d17-8a376b62b92e",
```

```
            "maxParticipants": 50,
            "minParticipants": null,
            "pricingCategoryList": {
                "nodes": [
                    {
                        "id": "ba9cc115-0eca-40f1-a953-caebb8e7695c",
                        "label": "Adult",
                        "minParticipants": null,
                        "maxParticipants": null,
                        "maxParticipantsDepends": null,
                        "units": 0,
                        "unitPrice": {
                            "netFormattedText": "€ 136.40",
                            "grossFormattedText": "€ 136.70"
                        },
                        "totalPrice": {
                            "grossFormattedText": "€ 0.00"
                        }
                    },
                    {
                        "id": "845a8e9c-1d22-4861-9821-5161b3938125",
                        "label": "Child",
                        "minParticipants": null,
                        "maxParticipants": null,
                        "maxParticipantsDepends": null,
                        "units": 0,
                        "unitPrice": {
                            "netFormattedText": "€ 85.30",
                            "grossFormattedText": "€ 85.40"
                        },
                        "totalPrice": {
                            "grossFormattedText": "€ 0.00"
                        }
                    }
                ]
            }
        }
    }
}
```

We will observe that there are two pricing categories for this option. They are

- Adult
- Child

Note that the units for each pricingCategory are initially set to zero.

We must now present these pricingCategories to the consumer and allow them to modify the value for units.

There will typically be are also some restrictions on the values that can be set as determined by each of the following

- overall
  - **minParticipants** - If set, you are required to book at least this number of PAX regardless of their category.
  - **maxParticipants** - if set you are required to book no more than this number of PAX regardless of their category.
- per category
  - **minParticipants -** if set you are required to book at least this number of the given category.
  - **maxParticipants -** If set, you are required to book no more that the given number of this category
  - **maxParticipantsDepends** - If set, this will indicate a relationship between categories such as the ration of Child to Adult and you must honour this requirement

It is important to ensure that the consumer can not breach these restrictions as doing so will result in an invalid availability that can not be added to a booking.

In order to set the values for units We will make one or more calls to the Availability to set the values.

5.5.2 Setting the units for each requested Pricing Category

5.5.2.1 Send the units to the availability

Send the required units for each pricingCategory

```
query  {
    availability(
        id: $availabilityId
```

```
        input: {
            pricingCategoryList: [
                { id: $pricingCategoryId1, units: $pricingCategoryUnits1 }
                { id: $pricingCategoryId2, units: $pricingCategoryUnits2 }
            ]
        }
    )
    {
    totalPrice {
        grossFormattedText
    }
    isValid
  }
}
```

5.5.2.2 Example Response

Check the total proce and the validity of the availability

```
{
    "data": {
        "availability": {
            "totalPrice": {
                "grossFormattedText": "€ 358.80"
            },
            "isValid": true
        }
    }
}
```

We will observe that the total gross formatted price has been updated to reflect the units we have set and that the `isValid` has returned as true indicating that the options we have requested are valid.

It is important to note that the total price returned may not be the sum and multiplication of the initial per category prices that where initially observed. This is because some products offer volume discounts and so the per category price for certain categories may have been reduced. There are additional fields available on the Availability query that can be used to return this information to the consumer

5.6 Creating a booking

Now that you have a valid Availability object you must create a Booking and then add the Availability to this booking before you can begin the checkout flow.

We create a booking with a call to the bookingCreate mutation

5.6.1 Create a new Booking

5.6.1.1 Mutation

Create a new Booking (cart)

```
mutation {
  bookingCreate (
        input: {
            autoFillQuestions: true
        }
    )
  {
    id
    state
    isComplete
    paymentState
  }
}
```

5.6.1.2 Example Response

Retrieve the id and other status information

```
{
    "data": {
        "bookingCreate": {
            "id": "ffd05542-f38e-446b-9861-35ecd389c3b3",
            "state": "OPEN",
            "isComplete": false,
            "paymentState": "ON_ACCOUNT"
        }
    }
}
```

We strongly recommend passing the autoFillQuestions = true option to the bookingCreate mutation. Many suppliers will configure optional questions on a product or on the checkout flow and these can present additional friction for the consumer. By using the autoFillQuestions options we are asking the system to bypass these additional questions resulting in less details for the consumer to complete.

The response will contain the unique booking ID just created along with any other state information we have requested.

This booking does not yet include our opinionated availability and so the next step is to add the availability to the booking by making a call to the bookingAddAvailability mutation

5.6.2 Create a new Booking with Consumer Trip

We also support the association of a Consumer Trip on booking with optional supporting fields for both the ConsumerTrip and the Consumer . See below for a basic example

5.6.2.1 Mutation

Create a new Booking (cart)

```
mutation {
  bookingCreate (
    input: {
      consumerTrip: {
        selector: {
          type: "reference",
          value: "<STRING REFERENCE>"
        }
      }
    }
  )
  {
    code
    consumerTrip{
      id
    }
  }
}
```

5.6.2.2 Example Response

Retrieve the id and other status information

```
{
  "data": {
    "bookingCreate": {
      "code": "BXJJWN",
      "consumerTrip": {
        "id": "831c4974-044d-4591-af55-d3c45756e8e0"
      }
    }
  }
}
```

5.6.3 Add your Availability to your Booking

5.6.3.1 Request

Add the availability to the booking (cart)

```
mutation {
  bookingAddAvailability(
    input: {
      bookingSelector: {id: $bookingId},
      id: $availabilityId
    }
  )
  {
    isComplete
  }
}
```

5.6.3.2 Example Response

Check the status of isComplete

```
{
    "data": {
        "bookingAddAvailability": {
            "isComplete": false,
```

```
            }
        }
    }
```

We will observe that `isComplete = false`. This will always be the case for a newly created booking and indicates that there are unsatisfied questions on the booking that we must respond to before it is possible to commit the booking

5.7 Input answers to the booking

In a process similar to that used to update the Availability you must now:-

- Retrieve full details of the booking's questions
- Answer each question that is present
- Iterate the process until `isComplete = true`

A booking has two optional static values that we recommend you always query and respond to. These are

- **leadPassengerName** - You may set this to any string value. We recommend that you do so in the format "<familyName>, <givenName>" and that you capitalise the <familyName>
- **partnerExternalReference** - You may set this to any string. We recommend that you use this to store your own unique reference for this booking. You may use this value later to retrieve a booking based on your value

In addition there are likely to be one or more booking questions that must be answered. Questions may exist at three different levels within the booking. These are

- **The Booking** - These questions apply to the whole of the booking.
- **Each Availability** - These questions apply to the whole of the availability. As it is possible to add more than one availability to a booking there may be different questions contained in each availability.
- **Each Person within each Availability** - These quesrions relate to each individual added to the given Availability. If say, you have added 2 Adults and one Child to the booking there may be questions for each of the three individuals and each must be presented to the consumer and the answes submitted to the system.

5.7.1 Structure of a question list

5.7.2 Request

Request the data for a questionList

```
questionList {
  nodes {
    id
    label
    type
    dataType
    dataFormat
    answerValue
    isRequired
  }
}
```

5.7.3 Example Response

The returned data

```
"nodes": [
  {
    "id": "203be336-b549-4779-8365-784895913c88",
    "label": "First name",
    "type": "NAME_GIVEN",
    "dataType": "TEXT",
    "dataFormat": null,
    "answerValue": null,
    "isRequired": true
  }
]
```

The following values are queried and returned for each question

- **id** - a unique id for this question that will be used when providing a response
- **label** - the label to be displayed to the consumer. Labels will be returned in the requested language
- **type** - an enumeration for the question type

- **dataType** - the type of data that is expected [TEXT, DATE, OPTIONS etc]
- **dataformat** - the restrictions on the format of the data that must be returned eg ["EMAIL", "PHONE"]
- **answerValue** - the value of any answer that has already been provided
- **isRequired** - indicates that the question must be satisfied before the booking can be committed

We will now issue a query to retrieve all of the questions for the booking

5.7.4 Fetch the booking questions

5.7.4.1 Request

Fetch the booking Questions

```
query holibob($bookingId: String!) {
  booking(id: $bookingId) {
    id
    code
    leadPassengerName
    partnerExternalReference
    state
    isSandboxed
    paymentState
    partnerChannelBookingUrl

    questionList {
      nodes {
        id
        label
                 autoCompleteValue
                 type
        dataType
        dataFormat
        answerValue
        isRequired
      }
    }

    availabilityList {
      nodes {
        id
```

```
        date
        product {
          id
          name
        }
        questionList {
          nodes {
            label
            dataType
            dataFormat
            answerValue
            isRequired
          }
        }
        personList {
          nodes {
            id
            pricingCategoryLabel
            isQuestionsComplete
            questionList {
              nodes {
                label
                answerValue
              }
            }
          }
        }
      }
    }
  }
}
```

5.7.4.2 Example Response

Booking data with questionLists

```
{
    "data": {
        "booking": {
            "id": "f47e2915-1124-4482-875c-7ced18997403",
            "code": "PSMF4W",
            "leadPassengerName": null,
            "partnerExternalReference": null,
            "state": "OPEN",
```

```
"isSandboxed": true,
"paymentState": "REQUIRED",
"partnerChannelBookingUrl": "https://demo.booking.sandbox.holibob.tech",
"questionList": {
    "nodes": [
        {
            "id": "203be336-b549-4779-8365-784895913c88",
            "label": "First name",
            "autoCompleteValue": null,
            "type": "NAME_GIVEN",
            "dataType": "TEXT",
            "dataFormat": null,
            "answerValue": null,
            "isRequired": true
        },
        {
            "id": "2cb6b546-63f1-4d0f-af3c-39e14408890a",
            "label": "Last name",
            "autoCompleteValue": null,
            "type": "NAME_FAMILY",
            "dataType": "TEXT",
            "dataFormat": null,
            "answerValue": null,
            "isRequired": true
        },
        {
            "id": "00a2ed5c-de09-419e-a48b-7a8cd43b6067",
            "label": "Email Address",
            "autoCompleteValue": null,
            "type": "EMAIL_ADDRESS",
            "dataType": "TEXT",
            "dataFormat": "EMAIL_ADDRESS",
            "answerValue": null,
            "isRequired": true
        },
        {
            "id": "16fb1dd1-d4f5-4d4c-abe7-0ce150568e1e",
            "label": "Phone Number",
            "autoCompleteValue": null,
            "type": "PHONE_MAIN",
            "dataType": "TEXT",
            "dataFormat": "PHONE_NUMBER",
            "answerValue": null,
            "isRequired": true
        }
```

```
            ]
        },
        "availabilityList": {
            "nodes": [
                {
                    "id": "711a0648-215e-43f5-9d17-8a376b62b92e",
                    "date": "2023-04-07",
                    "product": {
                        "id": "e33751d7-485b-44a4-ad8f-2637ff749353",
                        "name": "Walk Through Test Product"
                    },
                    "questionList": {
                        "nodes": []
                    },
                    "personList": {
                        "nodes": [
                            {
                                "id": "3e4db2d9-d20d-414d-97c9-d50b100e9237",
                                "pricingCategoryLabel": "Adult",
                                "isQuestionsComplete": true,
                                "questionList": {
                                    "nodes": []
                                }
                            },
                            {
                                "id": "4b41ad76-d665-40a5-b70e-79c89dba75b5",
                                "pricingCategoryLabel": "Child",
                                "isQuestionsComplete": true,
                                "questionList": {
                                    "nodes": []
                                }
                            },
                            {
                                "id": "4b5bd9f2-3a8d-4e17-9839-1d4545d0b5ab",
                                "pricingCategoryLabel": "Adult",
                                "isQuestionsComplete": true,
                                "questionList": {
                                    "nodes": []
                                }
                            }
                        ]
                    }
                }
            ]
        }
```

```
        }
    }
}
```

We will observe in the example response that the questionList contained directly in the single availability node and in each of the person nodes is returned empty in this example but this will not be the case for many products and so it is important to always issue a full query to establish the full list of questions.

We are now in a position to present the nested list of questions to the consumer and seek their responses.

We will then send the consumers responses to the system

5.7.5 Send the Answers for each Booking Question

5.7.5.1 Request

Send answers to the system

```
query {
  booking(
    id: $bookingId
    input: {

      leadPassengerName: "BRYCE Graeme Allan",
      reference: "12345"

      answerList: [
        {questionId: $questionId1, value: "Graeme"}
        {questionId: $questionId2, value: "Bryce"}
        {questionId: $questionId3, value: "graeme@holibob.tech"}
        {questionId: $questionId4, value: "+447745056933"}
      ]
    }
  ) {
    canCommit
  }
}
```

5.7.5.2 Example Response

Check the value of canCommit

```
{
    "data": {
        "booking": {
            "canCommit": true
        }
    }
}
```

We will confirm that all questions on the booking have been satisfied by requesting the `canCommit` value on the response. When this is `true` we are able to proceed to commit the booking.

5.8 Commit Booking

Our booking is now fully populated with a single opinionatedAvailability and all of the answers to required questions but the booking remains in the OPEN state and has not yet been committed.

The final step in this process is to commit the booking

We must however consider the matter of payment.

Typically, an API partner will be configured for a payment state of "ON_ACCOUNT" and so any booking that has a fully satisfied set of options and questions can immediately be committed. This will cause the system to process the booking and complete the following steps

- Send the booking to the supplier
- Retrieve any tickets issued by the supplier
- Compile a single PDF voucher containing all details of the booking including the embedding of any tickets
- Optionally, subject to configuration, send a confirmation email to the Consumer or Booking Agent containing a copy of the voucher

> ⚠ It is assumed that if your API payment is set to ON_ACCOUNT then you will already have arranged to take payment from the consumer for the gross amount of the booking.
>
> When you make a call to the bookingCommit mutation you are confirming your obligation to pay for the Booking and your will be invoices in due course.

To commit a booking we simply make a call to the bookingCommit endpoint passing the `id` of the booking.

5.8.1 Commit the booking

5.8.1.1 Request

Commit the booking

```
mutation holibob($bookingId: String!) {
  bookingCommit(bookingSelector: {id: $bookingId })
  {
    state
    voucherUrl
  }
}
```

5.8.1.2 Example Response

Check the state of the booking and the voucherUrl

```
{
    "data": {
        "bookingCommit": {
            "state": "PENDING",
            "voucherUrl": null
        }
    }
}
```

We will observe that the status of the booking has changed to “PENDING” indicating that the system is performing all necessary steps to communicate with the supplier and compile the voucher. We will also observe that in the initial response, the `voucherUrl` is null.

There are several ways in which We will establish that a booking has completed processing. These are

- You provide a webhook that Holibob will call upon completion of the processing
- You poll the booking until such time as the status changes from “PENDING”

5.8.2 Poll for the state of the Booking

5.8.2.1 Request

Poll the state of the booking

```
mutation holibob($bookingId: String!) {
  bookingCommit(bookingSelector: {id: $bookingId })
  {
    code
    state
    voucherUrl
  }
}
```

5.8.2.2 Example Response

Check the state of the booking and the voucherUrl

```
    "data": {
        "booking": {
            "code": "ABC123"
            "state": "CONFIRMED",
            "voucherUrl": "https://s3.eu-west-1.amazonaws.com/tech.holibob.files/
bookingVouchers/9fd195cd-f814-4700-8940-79679e0713fe/voucher.pdf?X-Amz-Algorithm=AWS4-HMAC-
SHA256&X-Amz-Credential=AKIA4H5CGSAPISTGYLFI%2F20230410%2Feu-west-1%2Fs3%2Faws4_request&X-
Amz-Date=20230410T091131Z&X-Amz-Expires=10000&X-Amz-
Signature=24dbde04c1c865e7a24111a8d183d8b4244bb7aadffb2df0bd5baf14878d50d7&X-Amz-
SignedHeaders=host"
        }
```

```
    }
}
```

Note that for security reasons the voucherUrl is temporary and you should not persist the URL to you storage.

You can of course retrieve the PDF and persist this to storage.

If at any time you need a fresh URL for the voucher you can issue the same query again.

The query is able to return everything that is linked to the Booking, however, in this case you would not want such extensive data.

The query in this example requests the key details of the booking including the following:

- **code** - this is a unique consumer facing code that is equivalent to the ID of the booking and typically used when communicating with customer service representatives.
- **state** - indicates if the booking is OPEN, CONFIRMED, PENDING or FAILED.
- **voucherUrl** - a single use URL that may be used to retrieve the PDF of the voucher. The single PDF will have any tickets embedded within it as may be required by the supplier.

5.8.3 Retrieve all bookings for ConsumerTrip or Consumer

You can identify the consumer and consumerTrip that a booking relates to by using one of the following filters on the `bookingList` query

- **consumerTripExternalReference** - filter bookings related to a specific ConsumerTrip reference
- **consumerExternalReference** - filter bookings related to a specific Consumer reference

5.8.3.1 Request

BookingList for specific ConsumerTrip

```
query BookingList {
  bookingList(
    filter: { consumerTripExternalReference: "<STRING REFERENCE>" }
  ) {
```

```
    recordCount
    nodes {
      code
      id
      consumerTrip {
        id
        partnerExternalReference
        consumer {
          id
          partnerExternalReference
          familyName
        }
      }
    }
  }
}
```

5.8.3.2 Example Response

List of Bookings

```
{
  "data": {
    "bookingList": {
      "recordCount": 2,
      "nodes": [
        {
          "code": "BXJJWN",
          "id": "c65b7727-0442-4c3f-95cb-6fff980cb721",
          "consumerTrip": {
            "id": "831c4974-044d-4591-af55-d3c45756e8e0",
            "partnerExternalReference": "<STRING REFERENCE>",
            "consumer": {
              "id": "e4e314c2-8627-439c-bbd2-0ac4d1d80877",
              "partnerExternalReference": "yyyy-yyyy-yyyy-yyyy",
              "familyName": "smith"
            }
          }
        },
        {
          "code": "3FR4ZF",
          "id": "afdf25d4-76d1-4855-8bef-b866b56bc03e",
          "consumerTrip": {
            "id": "831c4974-044d-4591-af55-d3c45756e8e0",
```

```
            "partnerExternalReference": "<STRING REFERENCE>",
            "consumer": {
              "id": "e4e314c2-8627-439c-bbd2-0ac4d1d80877",
              "partnerExternalReference": "yyyy-yyyy-yyyy-yyyy",
              "familyName": "smith"
            }
          }
        }
      ]
    }
  }
}
```

6 The Holibob API - Product & ProductList

Fetch Product List - Pagination

All queries accept pagination parameters.

When not specified, the default is to return 20 records.

The following parameters can be specified:

pageSize - the number of records returned in a single page (max 5,000).

page - the page number to be returned (min 1).

You can also request that the query return the following nodes to assist with pagination UI:

totalRecords - the number of records available for the given query.

unfilteredRecords - the total number of records that would have been available if the filters were not applied

pages - the total number of pages available (the totalRecords / pageSize).

nextPage - the next page if available.

previousPage - the previous page if available.

$.store.books[*].author

123.5 kilobytes

123.5 kilobytes

123.5 kilobytes

123.5 kilobytes

6.1 Fetch Product List - Product Content List

The productList query can return a contentList

Fields include:

type - an enumeration stating the type of content and typically used to split the list and render different content in sections within the UI

name - a short name for the item (some content types will only return name)

description - a text description

desctiptionHtml - same as description but including sanitized HTML tags such as , <i> and <br/>.

ordinalPosition - a sequence number in the contect of the type, typically used to control the order in which items are rendered.

Fetch Product List - Trees

The productList query is used to obtain a filtered list of product.

In this example the most basic form of search filter has been used. A more complex query may pass additional filters as lists of CITY, CATEGORY, ATTRIBUTE and more. See the full docs for the filter types available.

In the response, each product will contain the fileds that have been requested in the query.

ALL PRODUCT and indeed all entities of any type are returned with a unique 36 character UUID that may be used in subsequent queries to get more details or to reference the object when, for instance, adding to a booking.

If you are operating in sandbox mode: Setting the search string to "holibob" will return only test product that can be used to explore different product types and the related availability and booking API calls required to complete a booking.

Try changing the search string to 'paris' to return all product matching that term.

The importance of availabilityType

The availabilityType returned by this query is important when understanding how to proceed.

types include

DATE - The product availability will be per day and must be booked for a specific date.

DATE_TIME - The product may return multiple availabilities per day and must be booked for a specific time slot.

PASS - the product is a pass and will be purchased on the date of booking and be available for consumption up to the given expiry date. A pass will not accept an availability date wen being added to a booking and will allways be valid from the date of purchase.

Guide Prices

The guidePrice returned on a product must be read in conjunction with the guidePriceCurrency.

This price will be an APPROXIMATE Adult price for the next near availability of the product and is provided only as a guide and a from price. The true price of a product may vary by date and other criteria and will be returned only when a call is made to the availabilityList query.

Product images

Generally, each product will have a single previewImage and may also have a collection of galleryImages These are supplied with a URL that may be used to present the image to the viewer.

6.2 Fetch Product List - by Search

A generic filter of search can be used to find all product that contain the given search term in any of the name, description or keywords for the product.

In this example the search filter has been used. A more complex query may pass additional filters as placeIds, categoryIds, attributeIds and more.

6.3 Fetch Product List - by Place

Filtering the productList by place is one of the most common filters to apply.

A place may be any of:

a City - e.g. Edinburgh, London, New York, Perth - (City is inclusive of town and village)

a Country - e.g. Scotland, England, USA

a Region - e.g.

"Corfu" - the island as distinct from Corfu the town,

"Umbria" - a region of central Italy

"Greater London" - London city and the surrounding area

You can pass either:

placeId - the identifier for the place.

placeName - an exact match for the place name.

It is recommended that you ALWAYS use an ID rather than a name. This is because there are many places around the world that share a name.

For example a filter using placeName for "Perth" will return product in both Perth, Scotland and Perth, Australia whilst filtering by the placeId will ensure the unique identification of the place.

Please see the documentation of the SearchBar for a convenient query to enable users to search by place names of all types and return the appropriate ID.

6.4 Fetch Product List - by isFeatured

It is possible to set manual and rule based featuring of product.

A filter can be used to retrun only the featured product. This filter can be combined with other for instance to return only the featured products for a given place.

In the example the filter will return only the features product for the named place.

Fetch Product List - by Geo Circle

Fetch Product List - by Geo Rectangle

6.5 Fetch Product List - by Geo Circle

Filtering the productList by place is one of the most common filters to apply.

A place may be any of:

a City - e.g. Edinburgh, London, New York, Perth - (City is inclusive of town and village)

a Country - e.g. Scotland, England, USA

a Region - e.g.

"Corfu" - the island as distinct from Corfu the town,

"Umbria" - a region of central Italy

"Greater London" - London city and the surrounding area

You can pass either:

placeId - the identifier for the place.

placeName - an exact match for the place name.

It is recommended that you ALWAYS use an ID rather than a name. This is because there are many places around the world that share a name.

For example a filter using placeName for "Perth" will return product in both Perth, Scotland and Perth, Australia whilst filtering by the placeId will ensure the unique identification of the place.

Please see the documentation of the SearchBar for a convenient query to enable users to search by place names of all types and return the appropriate ID.

6.6 Fetch Product List - by Geo Rectangle

Filtering the productList by place is one of the most common filters to apply.

A place may be any of:

a City - e.g. Edinburgh, London, New York, Perth - (City is inclusive of town and village)

a Country - e.g. Scotland, England, USA

a Region - e.g.

"Corfu" - the island as distinct from Corfu the town,

"Umbria" - a region of central Italy

"Greater London" - London city and the surrounding area

You can pass either:

placeId - the identifier for the place.

placeName - an exact match for the place name.

It is recommended that you ALWAYS use an ID rather than a name. This is because there are many places around the world that share a name.

For example a filter using placeName for "Perth" will return product in both Perth, Scotland and Perth, Australia whilst filtering by the placeId will ensure the unique identification of the place.

Please see the documentation of the SearchBar for a convenient query to enable users to search by place names of all types and return the appropriate ID.

Error: Couldn't connect to server

0 bytes

0 bytes

6.7 Fetch Search Bar

This would typically be used to return a small number of options to a UI where the user is entering an initial search at the root of a product discovery page.

The result includes multiple types of record each with an ID and Name. The type of record returned may be any of COUNTRY, REGION, CITY, CATEGORY, PRODUCT

The result can then be presented to the viewer and the corresponding IDs user in the filter node of the ProductList query.

In the example london is passed as the searchTerm.

We can see in the response that three CITY are matched and then followed with a number of PRODUCT.

Cities

Londonderry, Northern Ireland

London, England

New London, United States

Products

London Showboat dinner cruise

London Eye tickets

Big Ticket: London Eye + Sea Life London + Madame Tussauds

London In One Day Tour with Changing of the Guard And London Eye

Lunch Cruises in London

6.8 Fetch Suggested Products for Product/ProductList

The product and productList query can return a `suggestedProductList`

isMandated - a flag that causes the suggestion to cary more force when rendered to the consumer

product -The full productType object that has all the available fields as the parent, except for `suggestedProductList` type

product & productList queries

```
query ProductList {
```

```
  productList {
    nodes {
      id
      name
      suggestedProductList {
        nodes {
          isMandated
          product {
            id
            ...other product fields
          }
        }
      }
    }
  }
}

query Product($productId: String) {
  product(id: $productId) {
    id
    name
    suggestedProductList {
      nodes {
        isMandated
        product {
          id
          ...other product fields
        }
      }
    }
  }
}
```

6.9 Rate Limits

Your holibob API key is associated with a usage plan that provides rate limits and quota to protect you and us from runaway code.

If you receive a “Limit Exceeded” error and believe this to be an error then you may contact holibob and request a rate limit increase.

The default limits are:

- 2 requests per second with a burst capability to 10 requests per second
- 2,500 requests per day

6.10 Sandbox mode

The credentials initially supplied will be valid for the production environment but limited to the sandbox mode. We strongly encourage you to do all initial development against the production URL as detailed above.

In the API this can be confirmed by examination of the response from the welcome query as the isSandbox flag will be set to true.

On white label and agent portals, you will also see an info icon on your account button.

Whilst in this mode, you are free to use the production API and portals for all testing activities including the creation and confirmation of bookings. These bookings will automatically be sandboxed and as such, they will not be confirmed with suppliers and you will not incur any charge for the bookings. All functionality is available in this state and you will be gaining access to production product and live availabilities.

All testing is therefore done against the production system ensuring that all code you write is immediately production ready.

Once you have completed initial integrations you will be able to apply to the holibob commercial team to have the credentials upgraded to allow live bookings. Once updated, all bookings confirmed on the production API will be live, you will be charged and the booking will be confirmed with the supplier.

You will be able to see all bookings within the holibob hub when logged in with credentials linked to your partner account. When using hub you can switch between viewing sandboxed or live bookings.

There is an additional API endpoint available that is simply an alias against the production API, however, this API will enforce sandbox mode regardless of the state of your account

The sandbox API is available at: https://api.sandbox.holibob.tech/graphql

7 The Holibob API - Product Discovery

7.1 Introduction

The **Product Discovery Query** is an **AI-powered recommendation** endpoint which delivers intelligent, contextually relevant recommendations based on traveller trip information. This endpoint is designed to power entire user interfaces by recommending not only **products** but filters, search terms and more.

This endpoint is an **alternative** to our **Product List Query,** which is a more traditional product endpoint with complex filtering, sorting and pagination.

7.2 Design Principles

Product Discovery is different from traditional product list endpoints and follows these principles.

Principle	
Free Text vs Data	Product Discovery supports free text input, allowing queries to be written entirely in natural language, just like a human conversation. This flexibility makes the endpoint usable with many different applications, including use with Large Language Models (LLMs), enabling integration into AI-powered assistants or conversational UIs. For every free text field, you also have the option of passing the data directly. You cannot pass both.
Personalisation	Every element of the response is personalised based on the contextual data provided in the request.
Ordering	Product Discovery automatically orders all recommendations by their likelihood of appealing to the user. The order cannot be controlled explicitly by an input. Here are some examples of data our models use: ▪ Past product performance for this type of trip ▪ Product's distance from the user's accommodation ▪ Weather during travel dates ▪ Seasonal trends within the destination ▪ Real-time user clickstream data ▪ And more…

Pagination	Traditional pagination is **not supported**. This is because the model that orders products is continuously changing (for example, real-time user clickstream data will impact orders). Instead, clients can provide a list of **seen product IDs** to ensure that subsequent requests return new, unrepeated recommendations. We suggest creating an infinite-scroll or "load more" experience.
Selected Destination	All Product Discovery responses are generated **within the context of a destination**, and all products will be relevant to that destination. You can pass the destination in the inputs, or the system will select one for you. The API can also **suggest alternative destinations** that may also be of interest, and you can switch destinations using the inputs.

7.3 Inputs

Product Discovery takes the following inputs. Each input has an option to pass **free text** OR pass **data**.

```
{
  where: {...},
  when: {...},
  who: {...},
  what: {...},
  consumerTripSelector: {...}
}
```

Input Variable	Description	Schema	Example/s
where	Pass information about **where** the traveller is starting from. This can be an airport, a hotel, or a destination. This is **required** to determine the **destination** and the **recommended destination list**.	`where: { freeText: String, data: { destinationId } }`	`where: { freeText: "Paris, France" }` `where: { freeText: "London Gatwick Airport" }`

			`where: { data: { destinationId: "destination-id-001" } }`

when	Pass information about when the traveller is looking for things to do.	`when: { freeText: String, data: { startDate: DateTime, endDate: DateTime, duration: { min: Number, max: Number, unit: MINUTE | HOUR | DAY } } }`	`when: { freeText: "Christmas Holidays" }`
			`when: { freeText: "Tonight" }`
			`when: { data: { startDate: "2025-03-01T00:00:00.000Z", endDate: "2025-03-08T00:00:00.000Z", duration: {max: 90, unit: MINUTE} } }`

who	This is used to pass any information about who the traveller/s is.	`who: { freeText: String, data: { personList: [{ age: Number }], interestList: [String] }`	`who: { freeText: "Two adults and a young child interested in history" }`

		`}`	`who: {`
			`  freeText: "A stag do of 10"`
			`}`
			
			`who: {`
			`  data: {`
			`  personList: [{ age: 20 }, { age: 22 }, { age: 3 }],`
			`  interestList: ["History"]`
			`  }`
			`}`

what	This is used to pass any information about what the traveller is looking for.	`what: {`
		`  freeText: String,`
		`  data: {`
		`  tagIdList: [String],`
		`  searchTerm: String,`
		`  price: { min: Number, max: Number },`
		`  rating: { min: Number, max: Number }`
		`  }`
		`}`
			`what: {`
			`  freeText: "The most booked cooking experiences which are less than 2 hours long and under £30"`
			`}`
			
			`what: {`
			`  data: {`
			`  tagIdList: ["tag-id-001"],`
			`  searchTerm: "Cooking experiences",`
			`  price: { max: 30 },`
			`  rating: { min: 4 }`
			`  }`
			`}`

consumerTripSelector	This is used when we already have consumer trip data in our	`consumerTripSelector: {`	`consumerTripSelector: {`

	system and can use that to personalise the response.	`type: REFERENCE | ID`
		`value: String`
		`}`
			`type: REFERENCE`
			`value: "partner-id-001"`
			`}`
			
			`consumerTripSelector: {`
			`type: ID`
			`value: "holibob-id-001"`
			`}`

7.4 Output

```
{
  selectedDestination: {...},
  recommendedDestinationList: {...},
  recommendedTagList: {...},
  recommendedSearchTermList: {...},
  recommendedProductList(seenProductIdList: []): {...}
}
```

Output Variable	Description	Schema

selectedDestination	The selected destination.	`selectedDestination: {`
		`id: String,`
		`name: String,`
		`...`
		`}`

recommendedDestination List	Other destinations which are most relevant to the trip.	`recommendedDestinationList: {`
		`nodes: {`
		`id: String,`
		`name: String,`
		`weatherForecast: {`
		`date: Date,`

		`type: CLEAR_DAY | CLOUDY | FOG ...,`
		`temperature: {`
		`value: Number,`
		`unit: CELSIUS | FAHRENHEIT`
		`}`
		`}`
		`...`
		`}`
		`}`

recommendedTagList	Tags which are most relevant to the trip and can be used for filtering.
	e.g Most Likely to Sell Out
		`recommendedTagList: {`
		`nodes: {`
		`id: String,`
		`name: String,`
		`...`
		`}`
		`}`

recommendedSearchTermList	Search terms which are most relevant to the trip.
	e.g Covent Garden Food Tours
		`recommendedSearchTermList(count: number): {`
		`nodes: {`
		`searchTerm: String,`
		`...`
		`}`
		`}`

recommendedProductList	Products which the traveller is most likely to purchase.	`recommendedProductList(`
		`seenProductIdList: ["product-id-001"],`
		`count: number`
		`): {`
		`nodes: {`
		`id: String,`
		`name: String,`
		`...`
		`}`

```
}
```

7.5 Examples

7.5.1 1. Recommended Destinations

You can use this field to get recommended destinations based on trip information. In the `where` field, you can pass a destination or something more precise, like an airport or hotel address.

Request

```
query Example {
  productDiscovery(where: { freeText: "London Gatwick Airport" }) {
    recommendedDestinationList: {
      nodes {
        id
        name
      }
    }
  }
}
```

Response

```
{
  recommendedDestinationList: {
    nodes: [
      {
        id: "destination-id-001",
        name: "London"
      },
      {
        id: "destination-id-002",
        name: "Cambridge"
      },
      {
        id: "destination-id-003",
        name: "Oxford"
```

```
      }
    ]
  }
}
```

7.5.2 2. Recommended Search Terms

You can use this field to get recommended search terms based on trip information. This can then be passed into the `what.data.searchTerm` field.

Request

```
query Example {
  productDiscovery(
    where: { freeText: "London, England" },
    who: { freeText: "Young Family" },
    when: {
      data: {
        date: { start: "2025-03-01T00:00:00.000Z", end: "2025-03-08T00:00:00.000Z" }
      }
    }
  )
  {
    recommendedSearchTermList: {
      nodes {
        id
        name
      }
    }
  }
}
```

Response

```
{
  recommendedSearchTermList: {
    nodes: [
      { searchTerm: "London Eye" },
      { searchTerm: "Tower of London" },
      { searchTerm: "Harry Potter Experiences" },
      ...
```

```
    ]
  }
}
```

7.5.3 3. Recommended Products

You can ask for a list of recommended products for a trip.

Request

```
query Example {
  productDiscovery(
    where: { data: { destinationId: "destination-id-001" } },
    who: { freeText: "Young Family interested in History" },
    when: {
      data: {
        date: { start: "2025-03-01T00:00:00.000Z", end: "2025-03-08T00:00:00.000Z" }
      }
    }
  )
  {
    recommendedProductList: {
      nodes {
        id
        name
      }
    }
  }
}
```

Response

```
{
  recommendedProductList: {
    nodes: [
      {
        id: "product-id-001",
        ...
      },
      {
        id: "product-id-002",
```

```
        ...
      },
      {
        id: "product-id-003",
        ...
      }
    ]
  }
}
```

8 The Holibob API - Uploading file to booking question of type `file`

Below is the interim solution to support file upload for a booking. There is a specific question type of `FILE` that needs to be available on the product to support this functionality.

1. Ensure you have question of type `file` created via Holibob
2. Follow standard booking flow until you reach the questions for the booking
3. Execute the `/getBookingUploadUrl` mutation to retrieve a signed url to which you can upload a file (mutation has to be run for each booking question of type `file`)
   Note that you need to provide `bookingId` *and* `bookingQuestionId`

```
mutation getBookingUploadUrl($bookingId: ID!, $bookingQuestionId: ID!) {
    bookingQuestionUpload(bookingId: $bookingId, bookingQuestionId: $bookingQuestionId,
contentType: "application/pdf") {
        uploadUrl
    }
}
```

4. Upload a file by sending PUT request to `uploadUrl`. Examples with HTTP[1]:
   a. PDF Example

```
http PUT '[uploadUrlGoesHere]' \
 Content-Type:application/pdf \
 x-amz-acl:private < file.pdf
```

   b. Image Example

```
http PUT '[uploadUrlGoesHere]' \
 Content-Type:image/jpeg \
 x-amz-acl:private < image.jpeg
```

5. Answer the booking question with value of "true" and follow the standard flow to commit the booking
   Example answer below:

1. https://httpie.io/

```
{
  "questionId": "826f12ad-363a-40c4-a1e5-9856302dd342",
  "value": "true"
}
```

6. Once answer value is set to `true` you can query `answerFile` field to receive path to the file

```
query useBooking($id: String!, $input: BookingInput) {
  booking(id: $id, input: $input) {
      questionList {
          nodes {
              id
              answerFile # url to file
              answerValue
              answerFormattedText
          }
      }
  }
}
```

> Maximum file size is 10MB

9 The Holibob API - Cancellation Policies

9.1 Introduction

A product's cancellation policy defines the consequence to a consumer of cancelling a booking availability.

For information about **Expedia** product's cancellation policies please check this document.

9.2 Holibob API Relevant Fields

The following tables explain where in the Holibob API cancellation policies can be found.

9.2.1 Effective Refund Amounts

Query	Path	Description
`booking`	`booking.cancellationEffectiveRefundAmount`	This field returns the refund amount if the **entire** `Booking` was cancelled at that moment. This field is `null` if the `Booking` does not contain an `Availability` which can be cancelled.
`booking`	`booking.availabilityList.nodes[*].cancellationState.effectiveRefundAmount`	This field returns the refund amount if the **single** `BookingAvailability` was cancelled at that very moment. This field is `null` if the `BookingAvailability` can not be cancelled.

9.2.2 Effective Cancellation Policy

Query	Path	Description
`booking`	`booking.availabilityList.nodes[*].cancellationState.effectivePenalty`	This field returns the appropriate penalty from the penalty list if the `BookingAvailability` was cancelled at that very moment.

9.2.3 Cancellation Penalty Lists

Most products have a generic cancellation policy, which can be found on the Product query and is translated into the `BookingAvailability` when the booking is made.

Query	Path	Description
`product`	`product.cancellationPolicy.penaltyList`	This is the cancellation policy for the product. This may change as the supplier chooses to update it.
`booking`	`booking.availabilityList.nodes[*].cancellationPenaltyList`	This is the cancellation policy for the product at the time of booking. This will never change.

9.3 Understanding a Cancellation Policy

A Cancellation Policy contains a list of Cancellation Penalties

A product's cancellation policy is made of a list of cancellation penalties. Each cancellation penalty contains the data required to calculate:

1. the **range of cancellation dates** for which this penalty is applicable
2. the **refund amount** issued to the consumer

9.3.1 Cancellation Date Ranges

- A cancellation penalty contains a `duration` field formatted in the ISO 8601[2]standard.
- The `duration` field can be used to calculate the range of cancellation dates applicable to that penalty.
- The `relativeTo` field dictates what date the `duration` is measured from and can take the value of `TRAVEL_DATE` or `BOOKING_DATE`. The `relativeTo` field will always be the same for all cancellation penalties inside a cancellation policy.
- If a cancellation penalty has a `duration` of `null` then this penalty becomes the exception.

2. https://en.wikipedia.org/wiki/ISO_8601#Durations

9.3.2 Refund Amount

- the `type` field dictates whether the refund amount is an absolute value or calculated as a percentage of the price paid. It can take the value of `PERCENTAGE` or `ABSOLUTE`.
- the `amountType` field dictates whether the amount is a refund or a charge and can take the value of `REFUND` or `CHARGE`.

9.3.3 Formatted Text

The data that powers cancellation policies is complicated so we generate a `formattedText` field at runtime in the language requested which can be used to present on the UI.

:check_mark:

9.4 Examples

In the following examples we refer to "Booking Date" and "Travel Date". These are specific to the BookingAvailability and can be found in the following places:

Booking Date	`booking.availabilityList.nodes[*].createdAt`
Travel Date	`booking.availabilityList.nodes[*].startAt`

9.4.1 1. Percentage Refund Relative to the Travel Date

9.4.1.1 Data

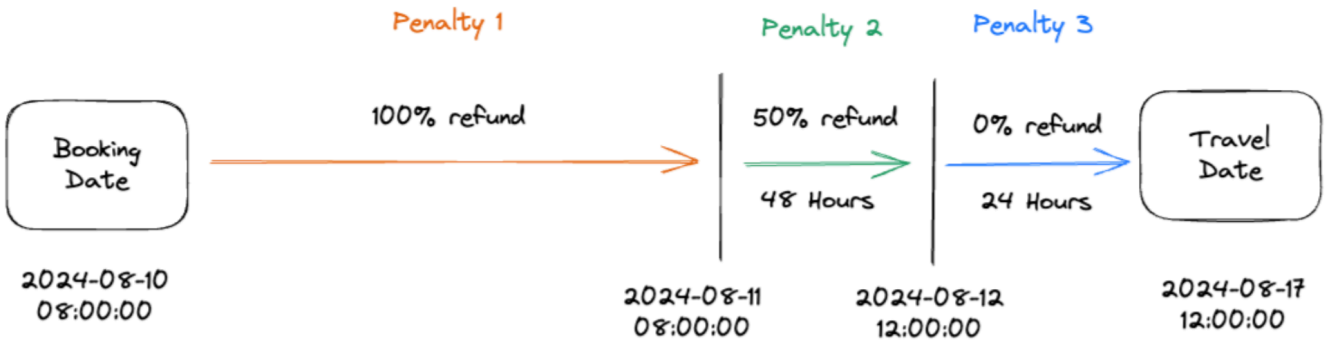
Booking Date	`2024-08-10 08:00:00 UTC`
Travel Date	`2024-08-17 12:00:00 UTC`
Cancellation Penalty List	

```
[
    {
        "type": "PERCENTAGE",
        "relativeTo": "TRAVEL_DATE",
        "duration": null,
```

```
            "refundPercentage": 100,
            "amount": null,
            "amountType": null,
            "amountCurrency": null
        },
        {
            "type": "PERCENTAGE",
            "relativeTo": "TRAVEL_DATE",
            "duration": "PT48H",
            "refundPercentage": 50,
            "amount": null,
            "amountType": null,
            "amountCurrency": null
        },
        {
            "type": "PERCENTAGE",
            "relativeTo": "TRAVEL_DATE",
            "duration": "PT24H",
            "refundPercentage": 0,
            "amount": null,
            "amountType": null,
            "amountCurrency": null
        }
    ]
```


9.4.1.2 Calculations

Cancellation Date Ranges



Penalty	Duration	Range of Cancellation Dates	Refund Amount
Penalty 1	`null`	From booking date than 48 hours before the travel date `2024-08-10 08:00:00 UTC` to `2024-08-15 12:00:00 UTC`	100%
Penalty 2	`PT48H`	From 48 hours before the travel date to 24 hours before the travel date `2024-08-15 12:00:00 UTC` to `2024-08-16 12:00:00 UTC`	50%
Penalty 3	`PT24H`	From 24 hours before the travel date to travel date `2024-08-16 12:00:00 UTC` to `2024-08-17 12:00:00 UTC`	0%

Free Cancellation / Full Refund

If this availability is cancelled before `2024-08-15 12:00:00 UTC` then the consumer has **free cancellation**.

Partial Refund

If this availability is cancelled between `2024-08-15 12:00:00 UTC` and `2024-08-16 12:00:00 UTC` then the consumer is issued a **partial refund**.

Non-Refundable

If this availability is cancelled after `2024-08-16 12:00:00 UTC` then this availability is **non-refundable**.

9.4.2 2. Absolute Refund Relative to the Booking Date

9.4.2.1 Data

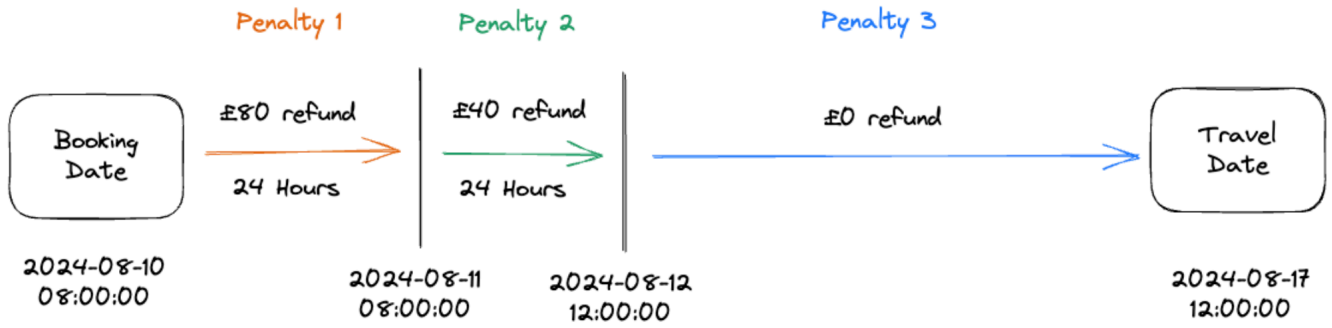
Booking Date	`2024-08-10 08:00:00 UTC`

Travel Date	`2024-08-17 12:00:00 UTC`
Total Price	`£80`
Cancellation Penalty List	


```
[
    {
        "type": "ABSOLUTE",
        "relativeTo": "BOOKING_DATE",
        "duration": "PT24H",
        "refundPercentage": null,
        "amount": 80,
        "amountType": "REFUND",
        "amountCurrency": "GBP"
    },
    {
        "type": "ABSOLUTE",
        "relativeTo": "BOOKING_DATE",
        "duration": "PT48H",
        "refundPercentage": null,
        "amount": 40,
        "amountType": "REFUND",
        "amountCurrency": "GBP"
    },
    {
        "type": "ABSOLUTE",
        "relativeTo": "BOOKING_DATE",
        "duration": null,
        "refundPercentage": null,
        "amount": 0,
        "amountType": "REFUND",
        "amountCurrency": "GBP"
    }
]
```

9.4.2.2 Calculations

Cancellation Date Ranges



Penalty	Duration	Range of Cancellation Dates	Refund Amount
Penalty 1	`PT24H`	From booking date than 24 hours after the booking date `2024-08-10 08:00:00 UTC` to `2024-08-11 08:00:00 UTC`	£80
Penalty 2	`PT48H`	From 24 hours after the booking date to 48 hours before the booking date `2024-08-11 08:00:00 UTC` to `2024-08-12 08:00:00 UTC`	£50
Penalty 3	`null`	From 48 hours after the booking date to travel date `2024-08-12 08:00:00 UTC` to `2024-08-17 12:00:00 UTC`	£0

Free Cancellation / Full Refund

If this availability is cancelled before `2024-08-11 08:00:00 UTC` then the consumer has **free cancellation**.

Partial Refund

If this availability is cancelled between `2024-08-11 08:00:00 UTC` and `2024-08-12 08:00:00 UTC` then the consumer is issued a **partial refund**.

Non-Refundable

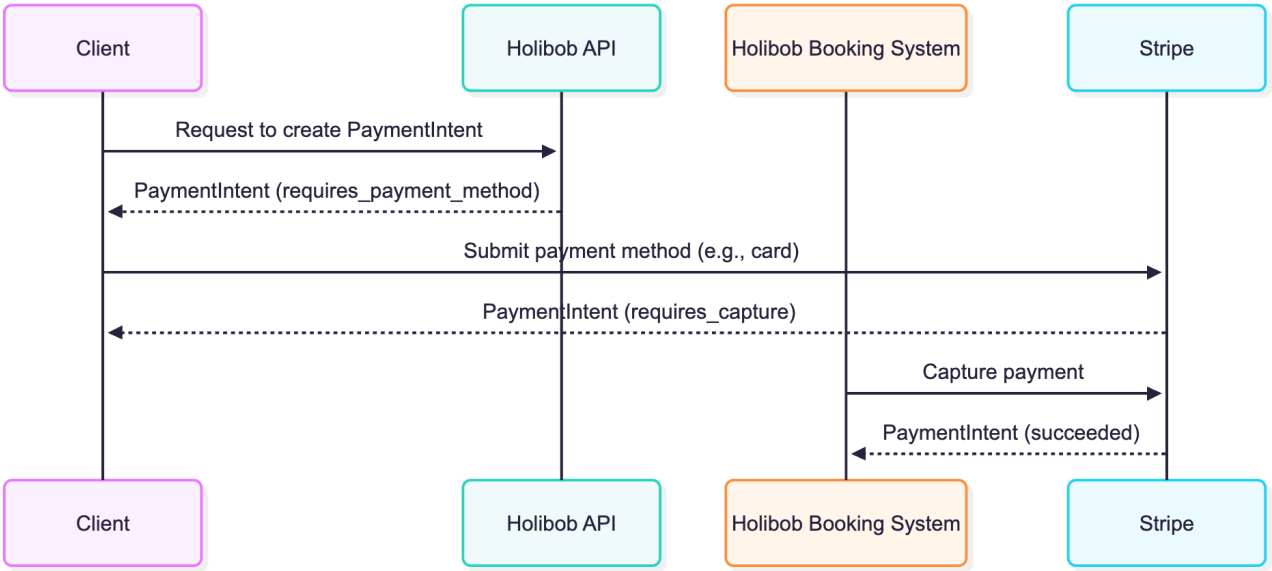
If this availability is cancelled after `2024-08-12 08:00:00 UTC` then this availability is **non-refundable**.

10 The Holibob API - Payment Flow

10.1 Introduction

To enable Holibob as the Merchant on Record, API partners must integrate their client site with the Stripe payment flow. This integration requires configuring the partner channel on the Holibob system with a payment type set to REQUIRED. The integration process involves two primary steps: fetching a Stripe Payment Intent via a GraphQL query and using this intent to render the Stripe payment form on the frontend. This document provides a detailed guide on implementing these steps to ensure seamless payment processing.

10.2 Stripe Payment Flow



10.3 Integration Guide

10.3.1 Overview

This guide explains how to integrate with Holibob's Stripe payment processing system. The integration involves two main steps:

1. Fetching a Stripe Payment Intent from our API
2. Using the Payment Intent to render the Stripe payment form on your frontend

10.3.2 Step 1: Fetching the Payment Intent

10.3.2.1 GraphQL Query

To get a Stripe Payment Intent, make a GraphQL query to our API:

```
query PaymentIntent($bookingSelector: BookingSelector!) {
  stripePaymentIntent(bookingSelector: $bookingSelector) {
    id
    amount
    clientSecret
    apiKey
    createdAt
  }
}
```

10.3.2.2 Parameters

`bookingSelector` : An object containing the booking ID

```
{
  id: string // The booking ID
}
```

10.3.2.3 Response

The response will contain:

- `id` : The Stripe Payment Intent ID
- `amount` : Amount to be charged in minor units

- `clientSecret` : Unique secret generated per Stripe intent
- `apiKey` : Stripe public key
- `createdAt` : When the payment intent was created

10.3.3 Step 2: Implementing the Payment Form

For the most up-to-date documentation, refer to the official Stripe docs linked below. Otherwise, you can continue by following our internal React integration guide.

> **Stripe Documentation**
> React Integration → https://github.com/stripe/react-stripe-js?tab=readme-ov-file
> Component Demo → https://checkout.stripe.dev/

10.3.3.1 Prerequisites

Install the required Stripe packages:

```
npm install @stripe/stripe-js @stripe/react-stripe-js
```

10.3.3.2 Implementation Steps

1. Initialise Stripe with the API key:

```
import { loadStripe } from '@stripe/stripe-js';

const stripePromise = loadStripe(apiKey);
```

2. Wrap your payment form with the Stripe Elements provider:

```
import { Elements } from '@stripe/react-stripe-js';

<Elements
```

```
  stripe={stripePromise}
  options={{
    clientSecret,
    locale: 'en' // Optional: Set the locale for Stripe Elements
  }}
>
  {/* Your payment form components */}
</Elements>
```

3. Create a payment form with the Card Element:

```
import { CardElement, useStripe, useElements } from '@stripe/react-stripe-js';

const PaymentForm = () => {
  const stripe = useStripe();
  const elements = useElements();
  const [error, setError] = useState(null);
  const [processing, setProcessing] = useState(false);

  const handleSubmit = async (event) => {
    event.preventDefault();

    if (!stripe || !elements) {
      return;
    }

    setProcessing(true);

    const result = await stripe.confirmCardPayment(clientSecret, {
      payment_method: {
        card: elements.getElement(CardElement)!,
      },
    });

    if (result.error) {
      setError(result.error.message);
      setProcessing(false);
    } else {
      // Payment succeeded
      if (result.paymentIntent.status === 'succeeded') {
        // Handle successful payment
```

```
      }
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <CardElement />
      {error && <div>{error}</div>}
      <button disabled={!stripe || processing}>
        {processing ? 'Processing...' : 'Pay'}
      </button>
    </form>
  );
};
```

10.3.4 Error Handling

If you attempt to commit a booking without satisfying the payment requirements, you will receive the following error:

`BookingCommitError: Cannot commit booking without paymentIntent`

10.3.4.1 Example Error Response

```
{
  "errors": [
    {
      "message": ""Cannot commit booking. Booking availability \"3MNZV2\" does not have
paymentIntent.",
      "extensions": {
        "code": "BOOKING_COMMIT_ERROR",
        "http": {
            "status": 400
        }
      }
    }
  ]
}
```

10.3.5 Sandbox vs Production

When bookings are made in sandbox mode, Holibob surfaces Stripe **test** credentials via the `stripePaymentIntent` GraphQL query. This allows you to simulate the full payment experience without charging real money.

In contrast, live bookings made in production will use live Stripe credentials, resulting in actual charges to customer cards.

10.3.5.1 Testing

- Use Stripe's test card numbers for testing:
  - Success: 4242 4242 4242 4242
  - Decline: 4000 0000 0000 0002
  - Insufficient: 4000 0000 0000 9995
  - 3D Secure: 4000 0025 0000 3155

11 The Holibob API - Booking Questions

When answering Booking Questions via the `booking` GraphQL query (The Holibob API - Look to Book Flow#Input-answers-to-the-booking (see page 39)):

1. All answer values must be sent as strings
2. All answer values must pass schema validation defined in the table below
3. To reset a value, you can pass `null`

DataType	Comments	DataFormat	Validation	Examples
`TEXT`		-	Minimum length 1	`John Smith`

`TEXT`		`PHONE_NUMBER`	Fully-qualified phone number (containing the area code and country code)
			We use google-libphonenumber[3]package for phone number validation. We expect E.164 format.
				`+12125551234`
				`+442079460958`
				`+61293744000`

`TEXT`		`EMAIL_ADDRESS`	Fully qualified email addresses.	`john.doe@example.com`
				`support@company.co.uk`
				`admin@subdomain.school.edu`

`TEXT`		`ADDRESS`	Minimum length 1	
`LONG_TEXT`	scheduled for deprecation for TEXT	-	Minimum length 1	
`INT`		-	Correctly casts to an integer value, between JavaScript `Number.MIN_SAFE_INTEGER` and `Number.MAX_SAFE_INTEGER`	0 123 123456

3. https://www.npmjs.com/package/google-libphonenumber

DataType	Comments	DataFormat	Validation	Examples
`DOUBLE`		-	Correctly casts to a float value, between JavaScript `Number.MIN_SAFE_INTEGER` and `Number.MAX_SAFE_INTEGER` or null	0 0.0001 12.003 -0.0001
`BOOLEAN`		-	Correctly casts to a boolean value	`true` `false`
`DATE`		-	Matches ISO 8601 standard (YYYY-MM-DD)	`2023-01-20`

`TIME`		-	Matches `HH:MM:SS[.s+]` format or null	`00:00:00`
				`09:52:31`
				`23:59:59.9999999`

`DATE_AND_TIME`		-	Matches ISO 8601 standard	`2020-01-20T00:00:00Z`
				`2020-01-20T00:00:00.123Z`
				`2020-01-20T00:00:00.123456Z`

`CHECKBOX_TOGGLE`	scheduled for deprecation for BOOLEAN	-	Correctly casts to a boolean value	`true` `false`
`ACCEPT`	scheduled for deprecation for BOOLEAN	-	Correctly casts to a boolean value	`true` `false`
`FILE`	Only for some INTERNAL products	`portableDocument` OR	Only accept "true"	`true`

DataType	Comments	DataFormat	Validation	Examples

		`image`
		OR
		`image,portableDocument`		

`OPTIONS`		-	Correctly matches the value of one of the `availableOptions` items	`GB`
				`US`
				`FR`
				
				`//availableOptions`
				`[`
				`{`
				`label: "United Kingdom",`
				`value: "GB",`
				`},`
				`{`
				`label: "United States",`
				`value: "US"`
				`},`
				`{`
				`label: "France",`
				`value: "FR",`
				`},`
				`];`

12 The Holibob API - Booking Cancellation

This section covers how to cancel bookings using the Holibob API. Cancellation allows you to cancel all booking availabilities under a booking.

12.1 Recommended Implementation Steps

1. Check Cancellation Eligibility
2. Display Cancellation Information
3. Execute Cancellation
4. Monitor Status

12.2 Retrieving Booking Cancellation Data

Before cancelling a booking, you should retrieve certain fields from the booking to add context data to display for cancellations and also to determine if cancellation is available for the booking. This information helps you present accurate cancellation details to users and determine whether the booking can be cancelled.

For detailed information about cancellation policies and how to use the cancellation data to determine when to enable cancellation functionality, refer to the Cancellation Policies documentation (see page 75).

12.2.1 GraphQL Query Example

Use the following query to retrieve the necessary cancellation context data:

12.2.1.1 Query

query

```
query Booking($bookingId: String) {
    booking(id: $bookingId) {
        canCancel
        cancellationEffectiveRefundAmount {
            amount
            currency
        }
        availabilityList {
```

```
            nodes {
                id
                state
                cancellationState {
                    effectivePenalty {
                        formattedText
                    }
                    effectiveRefundState
                    effectiveRefundAmount {
                        amount
                        currency
                    }
                }
            }
        }
    }
}
```

12.2.1.2 Response

response

```
{
    "data": {
        "booking": {
            "canCancel": true,
            "cancellationEffectiveRefundAmount": {
                "amount": 425,
                "currency": "USD"
            },
            "availabilityList": {
                "nodes": [
                    {
                        "id": "6fe66daf-23b0-466f-929e-cebd64bac4ab",
                        "state": "CONFIRMED",
                        "cancellationState": {
                            "effectivePenalty": {
                                "formattedText": "A full refund will apply if you cancel."
                            },
                            "effectiveRefundState": "FULL",
                            "effectiveRefundAmount": {
                                "amount": 425,
```

```
                                "currency": "USD"
                            }
                        }
                    }
                ]
            }
        }
    }
}
```

12.2.2 Key Fields Explained

- `canCancel` : Boolean field that indicates if the booking can be cancelled. Returns `true` if any of the booking availabilities under the booking are cancellable
- `cancellationEffectiveRefundAmount` : The total refund amount the customer will receive if the booking is cancelled
- `availabilityList.nodes.cancellationState` : Detailed cancellation information for each availability within the booking, including:
  - `effectivePenalty` : The effective penalty window that is applied to the booking.
  - `effectiveRefundState` : Denotes full, partial or no refund.
  - `effectiveRefundAmount` : The refund amount for this specific availability (for single Booking Availabilities, this should be the same as `cancellationEffectiveRefundAmount`)

12.3 bookingCancel Mutation

The `bookingCancel` mutation is used to cancel the entire booking. By doing so you are cancelling all the included booking availabilities with a supplier.

12.3.1 Input Parameters

The mutation accepts a `BookingCancelInput` object with the following fields:

Field	Type	Required	Description
`id`	String	✅ Required	The booking ID to cancel

Field	Type	Required	Description
`cancellationReason`	String	Optional	Reason for the cancellation

12.3.1.1 Mutation

query

```
mutation BookingCancel($input: BookingCancelInput!) {
    bookingCancel(input: $input) {
        id
    }
}
```

12.3.1.2 Response

response

```
{
    "data": {
        "bookingCancel": {
            "id": "3b8e81ba-aada-442d-bf53-d7dc4b63ff3d"
        }
    }
}
```

12.3.2 Asynchronous Processing

Like the `bookingCommit` mutation, `bookingCancel` is an asynchronous operation. After calling this mutation:

1. **Poll for Status Changes**: Monitor the booking state by periodically querying the booking to check for state changes. You should expect CONFIRMED → PENDING → CANCELLED
2. **Email Notifications**: If configured, the customer will receive a cancellation email
3. **Refunds**: If Holibob is the Merchant of Record, a refund will be automatically initiated to the customer

This documentation is part of the Holibob Partner API documentation suite. For additional support, contact the development team.